AmanAI Lab — Agentic AI Interview Prep Series Day 1 Complete Notes — Foundations, Concepts & Real Interview Questions

Series — Agentic AI Interview Prep | Day 1 of 5 | @AmanAI_lab

Day 1 Agentic AI Interview

1. What is Chatbot vs LLM vs Agentic AI vs AI Agents?

The One Line That Frames Everything

"A Chatbot responds. An LLM reasons. An AI Agent acts. Agentic AI is the spectrum that measures how much any system can act on its own."

Part 1 — Chatbot

What it is

A chatbot is the simplest form of AI interaction ever built. It does not think. It does not reason. It simply matches what you say to a pre-written response using rules, decision trees, or basic keyword retrieval.

It has no understanding of your actual goal. It only knows the pattern it was trained to match. The moment your input falls outside that pattern, it fails.

Key characteristics

No reasoning. No memory. No tools. No loop. Stops after one reply. Cannot adapt.

Real Example

You say — "Book me a flight to Delhi." Chatbot says — "Here is the link to our booking page."

That is it. Nothing happened. No flight was searched. No date was checked. No booking was made. It just pointed you to a link and stopped. You still have to do everything yourself.

When chatbots were enough

2010 to 2018 — FAQ bots, customer service deflection, basic menu navigation. They solved simple, repetitive, predictable problems cheaply.

Part 2 — LLM (Large Language Model)

What it is

An LLM is a massive neural network trained on billions of words of human text. It can genuinely understand language, reason through complex problems, write code, explain concepts, summarise documents, and generate human-quality text on virtually any topic.

This was a massive leap forward from chatbots. But it has one fundamental limitation — it is stateless and passive.

One prompt goes in. One response comes out. It stops. It cannot take any action in the real world. It cannot check whether its answer was correct. It has no memory of what happened before. It is a brilliant advisor sitting in a room with no phone, no computer, and no way to do anything except talk.

Key characteristics

Genuine reasoning. Language understanding. Chain-of-thought. But — no tools, no memory across steps, no loop, no action, completely stateless.

Real Example

You say — "Book me a flight to Delhi." LLM says — "You should visit MakeMyTrip or Google Flights, enter your travel dates, filter by price, and select a seat preference."

Excellent advice. But nothing actually happened. You still have to open the browser, search, compare, and book yourself. The LLM told you what to do but did absolutely nothing.

Where LLMs hit their ceiling

Any task that requires more than one step, any task that requires interacting with external systems, any task where the answer needs to be verified against real-world data, any task that needs to run automatically without a human driving every step.

Part 3 — Agentic AI

What it is

Agentic AI is not a product or a system. It is a concept — a spectrum that describes how autonomous, goal-directed, and independently capable any AI system is.

The word agentic comes from agency — the capacity to act independently and make decisions. So Agentic AI simply means AI that behaves with agency. AI that does not wait to be told every single step but figures out those steps itself.

The critical insight is that Agentic AI is a sliding scale, not an on/off switch. Every AI system sits somewhere on this spectrum — from zero autonomy to full autonomy.

The 5 Properties That Define How Agentic a System Is

Property 1 — Goal Directedness. The system works toward a defined end goal across multiple steps. It does not just respond to a single input and forget what it was doing.

Property 2 — Autonomy. The system decides its own next action without a human approving every step. The less human involvement needed between start and finish, the more agentic it is.

Property 3 — Multi-Step Reasoning. The system breaks a complex task into smaller steps and executes them in sequence. It plans, not just responds.

Property 4 — Environment Interaction. The system perceives what is happening around it from tool results, APIs, databases, and user feedback — and reacts to that information in real time.

Property 5 — Adaptation. The system changes its plan when something goes wrong or when new information arrives. It does not blindly follow a fixed script when reality does not match expectations.

The Agentic AI Spectrum — 5 Levels

Level 1 — Zero Agentic — Basic LLM call. One prompt, one response, full stop. Zero autonomy. Example — asking ChatGPT to explain a concept.

Level 2 — Slightly Agentic — RAG Pipeline. The system retrieves information from a database before answering. Two steps instead of one. Still stops after the answer. Example — a company knowledge base chatbot that searches internal documents before responding.

Level 3 — Moderately Agentic — Copilot Systems. The system reasons across full context, suggests actions, and adapts in real time. But a human approves every action before anything happens. Example — GitHub Copilot suggesting code. You still press Tab to accept every suggestion.

Level 4 — Highly Agentic — AI Agent. The system receives a goal and executes multiple steps — using tools, observing results, adjusting plans — with minimal human involvement. Example — an agent that researches laptops, compares specs, and creates a summary table all on its own.

Level 5 — Fully Agentic — Multi-Agent Systems. Multiple specialised AI agents collaborate, delegate tasks to each other, and complete an entire complex workflow end to end. Example — one agent researches, another writes, another schedules, another monitors results — all working together from a single instruction.

The simplest way to remember Agentic AI

The more an AI system can do on its own between your first instruction and the final result — the more agentic it is.

Part 4 — AI Agent

What it is

An AI Agent is where the Agentic AI spectrum maxes out in a single system. It is the concrete, buildable, shippable implementation of Agentic AI. It is an actual system that exists in code and infrastructure.

If Agentic AI is the concept and the philosophy — an AI Agent is the thing you actually build and deploy to production.

An AI Agent receives a goal, plans the steps to achieve it, uses tools to act on the real world, observes what happens, updates its understanding, and loops back to take the next step — completely autonomously — until the goal is done.

The 5 Components Every AI Agent Must Have

Component 1 — The LLM — The Brain. This is where all reasoning happens. The LLM reads the current goal, the full history of what has happened, and the latest observation — and decides what the next action should be. It does not act itself. It only decides. It is the mind, not the hands.

Component 2 — Tools — The Hands. Tools are functions the agent can call to interact with the real world. Web search, code executor, file reader, email sender, calendar checker, database query, API caller, calculator. Without tools the agent can only generate text. It cannot do anything real.

Component 3 — Memory — The History. Memory stores every action taken and every result observed in the current task. This full history is passed to the LLM on every iteration so it knows exactly where it is. Without memory every loop iteration starts from scratch and the agent cannot make progress.

Component 4 — The Orchestration Loop — The Engine. This is the while loop that keeps the agent alive and running. It manages the cycle of Think → Act → Observe, updates the context after every action, and checks whether the stop condition has been met after every iteration.

Component 5 — The Stop Condition — The Exit. This tells the agent when its job is done. Options include goal achieved, maximum iterations reached, confidence threshold crossed, tool error limit hit, or human approval required. Without a stop condition the agent runs forever and burns through your budget.

How an AI Agent Works — One Complete Real Example

Goal — "Find the top 3 data science job openings in Bangalore today and send me the links."

Iteration 1 — Think: I need to search for jobs. Act: Call web search tool — "data science jobs Bangalore 2025." Observe: 10 results returned.

Iteration 2 — Think: I need to read the top 3 in detail. Act: Call web fetch tool on the top 3 URLs one by one. Observe: Full job descriptions returned for all 3.

Iteration 3 — Think: I have everything I need. Format the 3 links with job titles and company names. Act: Call email tool with the formatted message. Observe: Email sent successfully.

Goal achieved. Loop exits. Result returned to user.

Three iterations. One user instruction. Zero human involvement in between.

Three Production-Level AI Agent Examples

Example 1 — Customer Support Agent. Goal — resolve a wrong order complaint. The agent reads the complaint, calls the order management API, calls the inventory API, decides to offer a replacement, calls the email tool to send the resolution, and updates the CRM. Done without a single human agent involved.

Example 2 — Research Agent. Goal — write a 500 word market analysis of the EV industry in India. The agent searches for recent news, fetches 5 articles, extracts key data points, fetches government policy documents, synthesises everything, and writes the final analysis. Done in 3 minutes instead of 3 hours.

Example 3 — Code Review Agent. Goal — review this pull request and flag bugs or security issues. The agent reads every changed file, runs static analysis tools, checks for vulnerability patterns, searches documentation, writes detailed review comments, and posts the review to GitHub — automatically on every single PR.

The Full Comparison — One Table

	Chatbot	LLM	Agentic AI	AI Agent
What it is	Rule-based system	Reasoning model	A concept / spectrum	A built system
Reasoning	No	Yes	Varies	Yes
Memory	No	No	Varies	Yes
Tools	No	No	Varies	Yes
Loop	No	No	Varies	Yes
Autonomy	Zero	Zero	Spectrum	High
Acts on world	No	No	Varies	Yes
Example	FAQ bot	ChatGPT	RAG to Multi-Agent	LangGraph bot

The Closing Line — Say This in the Interview

"A chatbot is a pattern matcher — it responds without understanding. An LLM is a reasoning engine — it understands but cannot act. Agentic AI is the spectrum of autonomy — it describes how independently any AI system pursues a goal. An AI Agent is the full realisation of that spectrum — a system with a brain to reason, hands to act, memory to remember, and a loop to keep going until the job is actually done. The jump from LLM to Agent is not about making the model smarter. It is about giving the model the architecture to act."

2. What is the Agentic Loop — explain every stage, what the LLM does at each point, and what the whole system looks like when it is actually running?

"The Agentic Loop is the heartbeat of every AI agent. It is the continuous cycle of Perceive → Think → Act → Observe that keeps running — iteration after iteration — until the goal is completely done."

Why the Loop Exists — The Core Idea

An LLM on its own is a one-shot system. You give it a prompt, it gives you a response, and it stops. It has no awareness of whether its response was correct, no ability to take action, and no way to continue if the task needs more than one step.

The loop solves all three of these problems at once.

It gives the agent continuity — it remembers what happened before. It gives the agent feedback — it knows whether its last action worked. It gives the agent persistence — it keeps going until the job is actually done.

Without the loop there is no agent. The loop is not a feature of the agent. The loop is the agent.

The Full Loop — Every Stage in Complete Detail

Stage 1 — Perceive

What it is

Perceive is the input stage. The agent reads the current state of the world — everything it needs to know before it can make a decision.

What goes into Perceive

On Iteration 1 — the agent perceives only the original goal given by the user. Nothing else exists yet.

On every subsequent iteration — the agent perceives the full accumulated context. This includes the original goal, every action it has taken so far, and every observation it has received so far. The entire history is present.

What the LLM does here

Nothing yet. The LLM is not called at this stage. This stage is purely about assembling the context — taking the goal and the memory and packaging them together into a single prompt that the LLM will read in the next stage.

What breaks if Perceive is missing

The agent goes blind. It cannot see the goal. It cannot see what it has already done. Every iteration starts from zero with no idea what it is supposed to be doing or what has already happened. The agent becomes completely non-functional.

Stage 2 — Think

What it is?

Think is the reasoning stage. This is the only stage where the LLM is actually called. It reads the full context assembled in the Perceive stage and makes one decision — what is the single best next action to take right now to move closer to the goal?

What the LLM does here — in exact detail

Step 1 — The LLM reads the goal. It understands what success looks like.

Step 2 — The LLM reads the full history. It understands exactly where in the task it currently is.

Step 3 — The LLM reads the latest observation. It understands what just happened as a result of the last action.

Step 4 — The LLM asks itself — is the goal already achieved? If yes it outputs a final answer and signals the loop to stop.

Step 5 — If the goal is not yet achieved the LLM selects the next action from the available tools. It specifies exactly which tool to call and exactly what inputs to pass to it.

Step 6 — The LLM outputs a structured decision — tool name, input parameters, and its reasoning for why this is the right next step.

What breaks if Think is missing

The agent has no brain. It can perceive the world perfectly but it cannot decide what to do about it. Without Think the agent cannot plan, cannot select tools, cannot check whether it is done, and cannot produce a final answer. It is a body with no mind.

Stage 3 — Act

What it is

Act is the execution stage. The orchestrator takes the decision the LLM made in the Think stage and executes it in the real world by calling the selected tool with the specified inputs.

What happens here

The orchestrator reads the tool name and input parameters output by the LLM. It calls the corresponding function — web search, email sender, code executor, API caller, database query, file reader, or any other registered tool. The tool runs and produces a result.

What the LLM does here

Nothing. The LLM is not involved in Act at all. It already made its decision in the Think stage. The Act stage is pure execution by the orchestrator. This is an important distinction — the LLM decides, the orchestrator executes.

What breaks if Act is missing

The agent can think all it wants but it can never do anything. It becomes a very sophisticated planning machine that produces zero real-world results. It can decide to search the web a thousand times but if Act is missing nothing ever gets searched. The agent is all brain and no hands.

Stage 4 — Observe

What it is

Observe is the feedback stage. The result of the action taken in Stage 3 comes back — a search result, an API response, a code output, an error message, a confirmation, or any other output — and the agent reads it, evaluates it, and stores it in memory.

What happens here

The tool result is received. It is appended to the agent's memory as the latest observation. This observation becomes the new input for the next Perceive stage. The loop then goes back to Stage 1 with this updated context.

What the LLM does here

Nothing directly. The observation is stored in memory. The LLM will read and reason about it in the next Think stage. But at this moment the observation is simply being recorded.

What breaks if Observe is missing

The agent becomes amnesiac. It takes actions but never knows what happened as a result. It cannot learn from tool failures. It cannot build on previous results. It cannot know when it is getting closer to the goal. Every iteration is completely disconnected from the last. The agent will repeat the same actions forever because it never remembers what those actions produced.

The Stop Condition — The Fifth Element

The loop runs indefinitely unless something tells it to stop. That something is the Stop Condition — checked at the end of every Think stage.

There are four stop conditions every production agent needs:

Stop Condition 1 — Goal Achieved. The LLM determines that all parts of the goal are complete. This is the ideal exit — the agent did its job.

Stop Condition 2 — Maximum Iterations Reached. The agent has run a preset number of iterations — say 20 — without completing the goal. The loop exits to prevent infinite execution and runaway costs.

Stop Condition 3 — Tool Error Limit Hit. A tool has failed a preset number of times — say 3 consecutive failures. The loop exits gracefully rather than hammering a broken tool forever.

Stop Condition 4 — Human Approval Required. The agent has reached a decision point that requires human confirmation before proceeding — such as sending an email, making a payment, or deleting data. The loop pauses and waits.

Without at least Stop Condition 1 and 2 the agent will run forever.

What the Complete Running System Looks Like

Here is the full picture of an agent mid-run — every component active at the same time:

The User gives a goal — it enters the system once and stays as the anchor for every iteration.

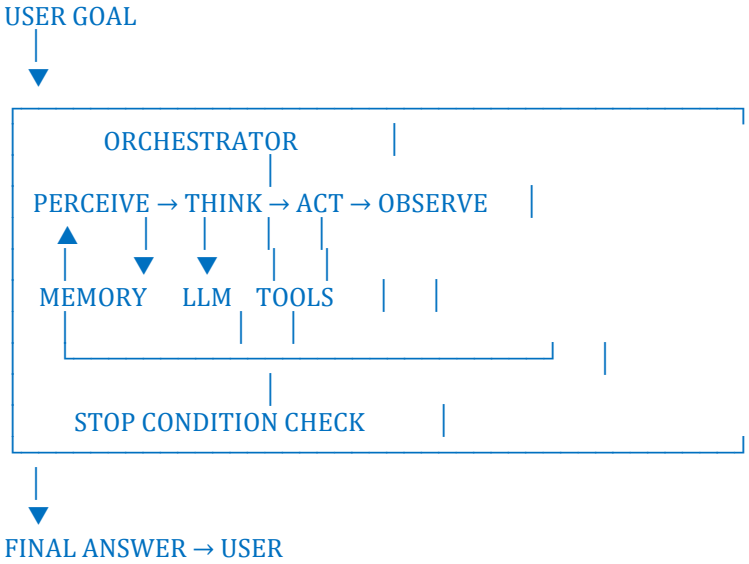
The Memory module holds the growing history — every action and every observation appended in real time. It gets longer with every iteration.

The Orchestrator runs the loop — it calls Perceive to assemble context, calls the LLM for Think, executes the tool for Act, stores the result for Observe, and checks the stop condition. It is the conductor of the entire system.

The LLM sits inside the Think stage — called once per iteration, reads the full context, outputs the next action decision.

The Tool Layer sits inside the Act stage — called once per iteration, executes the action the LLM decided on, returns the result to Observe.

The whole system in one visual flow:



One Complete Real Example — Every Iteration Shown

Goal — "Find me the best GenAI course available online right now and send me the link."

Iteration 1 Perceive — Goal only. No history. Think — I need to search for GenAI courses online. Call web search. Act — Web search tool called: "best GenAI course online 2025." Observe — 10 results returned. Top results include Coursera, DeepLearning.AI, and Udemy options. Stored in memory.

Iteration 2 Perceive — Goal + search results in memory. Think — I need to compare the top 3 options in detail. Fetch each one. Act — Web fetch tool called on top 3 URLs one

by one. Observe — Full course details, ratings, prices, and syllabi returned for all 3. Stored in memory.

Iteration 3 Perceive — Goal + search results + full course details in memory. Think — I have enough information. DeepLearning.AI has the highest rating and most relevant syllabus. This is the best recommendation. Goal is achieved. Prepare final answer. Act — Format the recommendation with the course name, link, rating, and one line reason. Observe — Final answer formatted successfully.

Stop Condition 1 triggered — Goal Achieved. Loop exits. Final answer returned to user.

Three iterations. One instruction. Complete answer delivered.

The Closing Line — Say This in the Interview

"The Agentic Loop is not just a design pattern — it is what transforms a passive language model into an active, autonomous system. Perceive gives it awareness. Think gives it intelligence. Act gives it capability. Observe gives it feedback. And the Stop Condition gives it judgment about when enough is enough. Remove any single one of these and you do not have a slower agent — you have no agent at all."

3. What makes an LLM "agent-ready" — which specific capabilities must it have before you can wrap it in an agent architecture?

The One Line Opening

"Not every LLM can be turned into an agent. Agent-readiness comes down to four non-negotiable capabilities — and if your model fails even one of them, your agent will break in production."

Capability 1 — Instruction Following

What it means

The LLM must follow precise, structured system prompt instructions reliably and consistently every single time. In an agent loop the orchestrator expects output in a specific format — usually structured JSON with a tool name and parameters. If the LLM drifts from this format even occasionally, the orchestrator cannot parse the decision and the Act stage never fires.

Why it is non-negotiable

The orchestrator is a piece of code. It does not understand natural language responses. It reads structured output only. If the LLM adds extra commentary, changes the format, or decides to respond conversationally instead of structurally — the entire loop breaks at that iteration.

What breaks without it

The agent produces beautifully reasoned responses that the orchestrator cannot read. The tool is never called. The loop dies on Iteration 1.

Real test

Give the LLM a strict output format rule. Run it 50 times on varied inputs. You need 95 percent or above compliance rate. Anything below that is not production safe.

Capability 2 — Tool Use and Function Calling

What it means

The LLM must understand that it has a registered set of tools, select the correct one for the current situation, and specify the exact input parameters that tool requires — all from reading the context alone.

This is called function calling or tool use. It is the bridge between the Think stage and the Act stage. Without this bridge the agent can reason all it wants but never actually do anything.

Why it is non-negotiable

The entire Act stage depends on this one capability. If the LLM cannot reliably select the right tool and format the correct inputs, the orchestrator calls the wrong function, passes garbage inputs, receives garbage results, and every downstream iteration reasons on corrupted data.

What breaks without it

The agent calls the wrong tool, passes malformed parameters, or hallucinates a tool that does not exist in the registry. Every downstream iteration is built on a broken foundation and the final answer is meaningless.

Real test

Give the LLM a goal and a list of 5 tools. Check whether it selects the correct tool on the first attempt, whether the parameter format exactly matches the tool schema, and whether it handles ambiguous situations without hallucinating a non-existent tool.

Capability 3 — Long Context Reasoning and Multi-Step Memory

What it means

As the agent runs — iteration after iteration — the context window grows. Every action taken and every observation received is appended to memory and passed back to the LLM on the next Think call. By Iteration 10 the LLM may be reading thousands of tokens of history before making its next decision.

The LLM must reason accurately over this entire growing context — keeping the original goal in focus, not losing track of completed steps, and not repeating actions it has already taken.

Why it is non-negotiable

Multi-step tasks are the entire point of building an agent. If the LLM can only reason reliably over short contexts it will lose the goal, repeat completed steps, or contradict its earlier decisions as the context grows. The agent becomes completely unreliable on anything beyond the simplest two or three step tasks.

Critical warning

A large context window on paper does not guarantee reliable long context reasoning in practice. Many models that advertise million token context windows show severe performance degradation well before they reach that limit. Always test at the actual context lengths your agent will reach in production — not at the model's theoretical maximum.

What breaks without it

The agent forgets what it already searched. It calls the same tool twice with identical inputs. It loses track of the original goal by Iteration 6 and starts producing answers that contradict observations it received three iterations ago.

Real test

Simulate a 15-iteration task history and check three things — does the LLM still reference the original goal correctly on Iteration 15, does it avoid repeating steps it already completed, and is its final answer consistent with all prior observations in the history.

Capability 4 — Reasoning and Planning Under Uncertainty

What it means

Real production tasks are never clean. Tools fail. API responses are incomplete. The goal turns out to be ambiguous halfway through. An agent-ready LLM must be able to change its plan when something goes wrong, try an alternative approach, decide when it has enough information to proceed, and recognise when it is genuinely stuck and needs to escalate to a human.

This is the highest-level capability and the hardest to evaluate. It is what separates an LLM that can complete simple scripted tasks from one that can handle real production complexity.

Why it is non-negotiable

Production environments are unpredictable. An LLM that can only follow the happy path will fail the moment reality does not cooperate. In an agent loop that failure does not just produce a wrong answer — it executes wrong actions in the real world. The stakes are completely different from a chatbot giving a bad response.

What breaks without it

The agent hits a tool failure on Iteration 3, has no recovery strategy, and either hammers the same failing tool forever until the max iteration limit triggers, or produces a confident hallucinated answer built on incomplete data.

Real test

Deliberately introduce a tool failure at Iteration 3. Does the LLM recognise the failure from the observation, decide on an alternative approach, and continue toward the goal? Or does it repeat the failed call or immediately produce a confident wrong answer pretending the failure did not happen?

The Bonus Capability — Knowing When to Stop and Escalate

This is not always listed but it is critical for production agents.

An agent-ready LLM must know when it does not know enough to act. It must be able to say — I do not have enough information to complete this goal confidently, I need to ask the user for clarification, or I have reached a decision point that requires human approval before I proceed.

An LLM that always produces a confident answer regardless of how incomplete its information is will make confident wrong decisions inside an agent loop — and those wrong decisions get executed as real actions in the real world. A wrong email gets sent. A wrong API call gets made. A wrong database entry gets written.

The Four Capabilities — Quick Reference Table

Capability	What it enables	What breaks without it
Instruction Following	Structured tool call output the orchestrator can parse	Loop dies on Iteration 1
Tool Use / Function Calling	Correct tool selected with correct inputs	Wrong or hallucinated actions executed
Long Context Reasoning	Goal tracking and step memory across iterations	Agent loses goal, repeats steps, contradicts itself
Reasoning Under Uncertainty	Recovery from tool failures and unexpected results	Agent breaks or hallucinates on first surprise

---

Agent-Ready Models Right Now

Closed Source — Best for Production

GPT-4o and GPT-4o Mini — Industry standard for tool calling reliability. Best starting point for any production agent. GPT-4o Mini is cost-effective for high-volume workloads where tasks are well-defined.

Claude Opus and Claude Sonnet — Best for complex multi-step reasoning and planning. Superior at following nuanced system prompt instructions and maintaining coherent multi-step plans across long contexts.

Gemini series — Exceptional long context window support. Strong choice for tasks requiring very long agent traces or large document processing within the loop.

Open Source — Best for Self-Hosted Production

GLM-4.5-Air — Purpose-built for agent tool use and web browsing. Strong agentic performance with efficient serving.

Qwen3-Coder-30B — Best open source choice for agentic coding tasks.

Qwen3-30B-Thinking — Best open source choice for complex reasoning agents requiring deep multi-step planning.

Kimi-K2.5 — Strong long context support. Well suited for long agent traces and multi-step planning tasks.

The Production Rule

Multi-model routing is the pattern serious production teams use. Use cheaper, faster models for simple well-defined tasks. Use frontier models for complex reasoning-heavy tasks. Benchmark on your own tool schemas and task sequences — generic benchmarks do not predict agent performance on your specific workload.

The Closing Line — Say This in the Interview

"Agent-readiness is not about picking the biggest or most famous model. It is about verifying four specific capabilities against your actual workload — can it follow structured instructions reliably, can it select and call tools correctly, can it reason accurately over a long and growing context, and can it adapt when things go wrong. A model that passes all four tests on your specific tasks and tool schemas is agent-ready. A model that looks impressive on generic benchmarks but fails one of these four on your workload will break your agent in production — and in production, that break always comes at the worst possible moment."

4. What is the difference between a reactive agent and a proactive agent — and when would you choose one over the other in a real system?

The One Line Opening

"A reactive agent waits for something to happen and then responds. A proactive agent does not wait — it anticipates, plans ahead, and acts before it is told to. The difference is not intelligence — it is initiative."

Part 1 — Reactive Agent

What it is

A reactive agent only acts when it receives an external trigger. It has no internal drive to do anything on its own. Something must happen first — a user message, an API call, an event, a scheduled trigger — before the agent wakes up, runs its loop, completes the task, and goes back to waiting.

It is entirely stimulus-response driven. No stimulus, no response. No trigger, no action.

The mental model

Think of a reactive agent like a customer support representative sitting at a desk. They do nothing until a customer walks in. The moment a customer arrives they spring into action, solve the problem completely, and then sit back down and wait for the next customer. They never go looking for problems to solve on their own.

How it works internally

Trigger arrives → Agent wakes up → Perceive goal → Run loop → Complete task → Return result → Go back to idle state → Wait for next trigger.

The agent has no awareness of anything happening outside of its current active task. Once the task is done it has zero situational awareness until the next trigger arrives.

Key characteristics

Waits for external trigger before acting. Has no awareness between tasks. Executes one task at a time. Stops completely when task is done. Has no internal goals of its own. Cannot anticipate problems before they occur.

Real World Examples

Example 1 — Customer Support Agent. Sits idle until a customer submits a complaint ticket. The ticket is the trigger. The agent reads the complaint, calls the order management API, checks inventory, sends a resolution email, and closes the case. Then it goes back to waiting for the next ticket.

Example 2 — Code Review Agent. Sits idle until a developer opens a pull request on GitHub. The PR event is the trigger. The agent reads the changed files, runs static analysis, writes review comments, and posts them to GitHub. Then it goes back to idle until the next PR.

Example 3 — Data Pipeline Agent. Sits idle until a new data file lands in a specific folder. The file arrival is the trigger. The agent reads the file, transforms the data, loads it into the database, and sends a confirmation. Done until the next file arrives.

When to choose a reactive agent

Choose reactive when the task is clearly event-driven. When there is a well-defined trigger that always signals work needs to be done. When the agent should only act when explicitly asked to. When predictability and auditability matter more than initiative. When the cost of unnecessary actions is high.

Most production agents in use today are reactive. They are simpler to build, easier to debug, cheaper to run, and much easier to reason about from a safety perspective. If something goes wrong you always know exactly what triggered the bad behaviour.

Part 2 — Proactive Agent

What it is

A proactive agent does not wait to be told. It continuously monitors its environment, anticipates future needs, identifies problems before they become critical, and takes action on its own initiative — without waiting for an external trigger.

It has its own internal goals and its own internal drive. It is always on, always watching, always thinking about what might need to happen next.

The mental model

Think of a proactive agent like a chief of staff at a company. They do not sit and wait for the CEO to give them every task. They monitor calendars, anticipate upcoming deadlines, spot potential problems before they blow up, prepare materials before they are asked for, and flag issues before anyone else notices them. They are always one step ahead.

How it works internally

Agent is always running → Continuously monitors environment → Detects a condition or anticipates a future need → Decides on its own to act → Runs loop to completion → Returns to monitoring → Detects next condition → Acts again.

The key difference is the agent itself is the source of the trigger. It decides when to act based on what it is observing — not because an external system told it to.

Key characteristics

Self-triggered based on internal monitoring. Continuously aware of its environment. Can anticipate problems before they occur. Has its own internal goals beyond just completing assigned tasks. Acts without being explicitly asked. Carries significantly higher risk if goals or constraints are not defined precisely.

Real World Examples

Example 1 — Infrastructure Monitoring Agent. Continuously monitors server health metrics. The moment it detects CPU usage trending toward a critical threshold — before any alert fires — it proactively scales up resources, clears non-essential processes, and sends a warning to the engineering team. Nobody asked it to. It anticipated the problem and acted.

Example 2 — Sales Intelligence Agent. Continuously monitors news feeds, competitor websites, and social media. The moment it detects that a competitor has announced a price drop, it proactively drafts a competitive response brief, updates the CRM with relevant accounts that might be at risk, and flags the situation to the sales team — all before anyone on the team even knows it happened.

Example 3 — Personal Productivity Agent. Monitors your calendar, email, and task list continuously. Two days before an important meeting it proactively researches the attendees, prepares a briefing document, checks whether any prep tasks are behind schedule, and sends you a readiness summary — without you asking for any of it.

When to choose a proactive agent

Choose proactive when the value comes from catching things early. When the cost of missing a signal is higher than the cost of acting unnecessarily. When the environment changes fast and waiting for explicit triggers means always reacting too late. When the

agent has a very well-defined scope and constraints so its autonomous initiative does not cause unintended side effects.

Part 3 — The Real Difference Between the Two

This is what interviewers actually want to hear.

The difference between reactive and proactive is not about how smart the agent is or how powerful the LLM is. Both can use the same LLM, the same tools, and the same loop architecture.

The difference is where the trigger comes from.

In a reactive agent the trigger comes from outside — a user, an event, a scheduled job, an API call. The agent is passive between triggers.

In a proactive agent the trigger comes from inside — the agent's own monitoring logic detects a condition and decides to act. The agent is never truly passive.

This distinction has enormous implications for safety, cost, and system design. A reactive agent that misbehaves is easy to stop — just do not send it a trigger. A proactive agent that misbehaves is much harder to stop because it generates its own triggers internally.

Part 4 — The Hybrid Pattern — What Most Production Systems Actually Use

In practice most serious production systems use neither pure reactive nor pure proactive. They use a hybrid pattern.

The monitoring layer is proactive — it continuously watches the environment and detects conditions.

The action layer is reactive — before the agent takes any significant action it checks in with a human or a rule-based approval system.

This gives you the awareness and early detection of a proactive agent with the safety and predictability of a reactive agent.

Example — An infrastructure agent proactively detects a potential outage condition. But before it scales resources or restarts services it sends a Slack message to the on-call engineer saying "I detected X condition and plan to take Y action — approve or deny." The engineer replies. Then the agent acts. Proactive perception, reactive execution.

This hybrid pattern is the safest and most practical design for production agentic systems right now.

The Comparison Table

	Reactive Agent	Proactive Agent
Trigger source	External — user or event	Internal — own monitoring
Idle behaviour	Completely inactive	Continuously monitoring
Initiative	Zero — waits to be asked	High — acts without being asked
Safety risk	Low — easy to control	Higher — harder to stop
Cost	Lower — only runs when triggered	Higher — always running
Best for	Event-driven tasks	Anticipation and early detection
Complexity	Simpler to build and debug	Significantly more complex
Real world use	90 percent of current agents	Growing — monitoring and intelligence systems

---

The Closing Line — Say This in the Interview

"A reactive agent is triggered by the world — it acts when asked. A proactive agent triggers itself — it acts when it decides something needs to happen. The choice between them is not a technical choice — it is a design philosophy choice about how much initiative you want to give your system. In most production systems today the right answer is a hybrid — proactive enough to detect problems early, reactive enough to require human approval before taking consequential actions. Full proactive agents are powerful but they require extremely precise goal and constraint definitions, because an agent that acts on its own initiative with poorly defined boundaries will eventually do something you did not want it to do."

5. How does an AI agent decide when to stop — what are the different termination strategies and what happens if none of them trigger?

The One Line Opening

"An agent that cannot decide when to stop is not a production agent — it is a runaway process burning tokens, money, and potentially taking harmful actions indefinitely. Termination logic is not an afterthought. It is a core part of agent design."

Why Termination is Harder Than It Sounds

Most people building their first agent focus entirely on making it work — getting the loop running, tools calling correctly, observations feeding back in. Termination feels like a simple add-on. It is not.

The agent has no natural sense of "done." It does not feel tired. It does not notice it has been running for 40 iterations. It does not know that it just sent the same email three times. Without explicit termination logic built into the architecture the loop runs forever — or until your API budget runs out.

In production this is not a theoretical problem. It is one of the most common failure modes teams hit when they first deploy agents at scale.

The 5 Termination Strategies Every Production Agent Needs

Strategy 1 — Goal Achievement Detection

What it is

This is the ideal exit. At the end of every Think stage the LLM evaluates one question — has the original goal been fully achieved based on everything observed so far? If yes it outputs a final answer and signals the orchestrator to exit the loop.

How it works

The system prompt explicitly instructs the LLM to check goal completion at the end of every reasoning step. The LLM reads the original goal, reviews the full observation history, and determines whether all parts of the goal have been satisfied. If satisfied it outputs a special completion signal — usually a structured JSON flag like `{"status": "complete", "final_answer": "..."}` — that the orchestrator reads and uses to exit.

What makes it reliable

The goal must be defined precisely and unambiguously at the start. Vague goals produce vague completion checks. If the goal is "research AI trends" the LLM can never be certain it is done because the goal has no clear success condition. If the goal is "find the top 3 AI funding rounds from this month and return them as a structured list" the LLM can check completion precisely.

What breaks it

Vague goals. Ambiguous success conditions. Goals that are too broad. If the goal cannot be checked against a clear condition, Strategy 1 fails and you must rely on the other strategies as safety nets.

Strategy 2 — Maximum Iteration Limit

What it is

A hard ceiling on the number of loop iterations the agent is allowed to run. The orchestrator counts iterations and forcibly exits the loop when the count reaches the preset maximum — regardless of whether the goal has been achieved.

How it works

The orchestrator maintains a simple counter. Every time the loop completes one full cycle — Perceive, Think, Act, Observe — the counter increments by one. When the counter hits the maximum the orchestrator stops calling the LLM, returns whatever partial result exists, and flags the task as incomplete.

Why it is non-negotiable

This is your safety net for every situation where Strategy 1 fails. Goal detection fails on vague goals. Tool failures can send the agent into confused loops. LLM reasoning errors can cause the agent to pursue the wrong sub-goal indefinitely. The max iteration limit catches all of these cases unconditionally.

How to set the right limit

During development run your agent on representative tasks and observe how many iterations it typically takes to complete them successfully. Set the maximum at roughly two to three times that number. High enough that normal tasks always complete. Low enough that runaway loops are caught quickly.

For simple tasks — 5 to 10 iterations. For medium complexity tasks — 15 to 25 iterations. For complex research or multi-step workflows — 30 to 50 iterations. Never set it to unlimited. Never.

What breaks it

Setting the limit too low — legitimate tasks get cut off before completion. Setting it too high — runaway loops waste significant compute and money before being caught.

Strategy 3 — Confidence Threshold

What it is

The LLM evaluates its own confidence in the current answer at the end of every Think stage. If its confidence crosses a preset threshold — meaning it is sufficiently certain the answer is correct and complete — it exits the loop without necessarily having used all available iterations.

How it works

The system prompt instructs the LLM to score its own confidence on a scale after every reasoning step. The orchestrator reads this confidence score. If it exceeds the threshold — say 0.85 out of 1.0 — the loop exits early with the current best answer.

Why it matters

Without this strategy the agent always runs until the max iteration limit even when it found the answer on Iteration 2. Confidence threshold allows early exit on easy tasks — saving tokens, reducing latency, and cutting costs significantly at scale.

The critical risk

LLM self-evaluation of confidence is not perfectly reliable. Models can be confidently wrong — they output a high confidence score on an answer that is actually incorrect. This strategy must always be used in combination with Strategy 1 and 2, never as the sole termination mechanism.

Strategy 4 — Tool Error Limit

What it is

A ceiling on how many consecutive tool failures the agent is allowed to encounter before the loop exits. If the tool the agent depends on is broken, returning errors, or timing out — there is no point running 47 more iterations hoping it will magically start working.

How it works

The orchestrator tracks consecutive tool failures per tool. If the same tool fails three times in a row — or if total tool failures across all tools exceed a preset number — the orchestrator exits the loop, returns a partial result, and flags the task as failed due to tool unavailability.

Why it matters

Without this strategy a broken API turns your agent into an infinite retry machine. Every iteration the LLM calls the broken tool. Every iteration the tool fails. Every iteration the LLM tries again. The loop runs until the max iteration limit — wasting every single one of those iterations on a tool that was never going to work.

What to set

Three consecutive failures on the same tool is a reasonable production default. After three failures the agent should either try an alternative tool if one exists, or exit gracefully with a clear error message explaining which tool failed and why the task could not be completed.

Strategy 5 — Human-in-the-Loop Pause

What it is

At certain predefined decision points the agent pauses the loop and waits for explicit human approval before continuing. This is not a termination strategy in the traditional sense — it is a controlled pause. But it is essential for any agent that takes consequential or irreversible actions.

How it works

The system prompt defines a set of high-stakes actions that require human approval — sending emails, making payments, deleting data, publishing content, calling external APIs with side effects. When the LLM selects one of these actions in the Think stage, instead of executing it immediately the orchestrator sends a notification to a human with the proposed action and waits for approval. If approved the loop continues. If denied the agent is instructed to find an alternative approach.

Why it matters

Some actions cannot be undone. A wrong email sent to a thousand customers cannot be unsent. A wrong payment cannot always be reversed. A deleted database record may not be recoverable. For these classes of action autonomous execution without human oversight is simply not acceptable regardless of how capable the agent is.

The production pattern

Most mature production agents use a tiered approval model. Low-stakes reversible actions — reading data, searching the web, running calculations — execute automatically. Medium-stakes actions — writing to a database, sending internal notifications — execute with logging but no approval needed. High-stakes irreversible actions — sending external emails, making payments, deleting records — pause for human approval every time.

What Happens If None of the Strategies Trigger

This is the scenario that should never happen in a well-designed system — but it does happen in poorly designed ones. Here is exactly what occurs:

The goal is too vague for Strategy 1 to ever detect completion. The max iteration limit was set to unlimited or was forgotten entirely. The confidence threshold was not implemented. Tools keep working so Strategy 4 never fires. No human-in-the-loop checkpoints were defined.

Result — the agent runs indefinitely. The LLM keeps reasoning, tools keep being called, observations keep accumulating, and the context window eventually fills up completely. When the context window overflows the LLM starts losing the beginning of its history — including the original goal. Now the agent is running without even knowing what it was originally trying to achieve. It hallucinates a new goal based on the most recent observations. It pursues that hallucinated goal. It never stops.

In production this translates to one of three outcomes. Your API costs spike to thousands of rupees before someone notices and manually kills the process. The agent starts taking unintended actions based on its hallucinated goal — sending wrong emails, making wrong API calls, writing wrong data. Or the system crashes when memory limits are hit.

This is not a theoretical edge case. It is a real failure mode that has happened in production systems. It is the reason every agent — no matter how simple — must have at minimum Strategy 1 and Strategy 2 implemented before it touches a production environment.

The Termination Design Checklist

Every agent you build must answer yes to all five of these before it goes to production:

Is the goal defined precisely enough for the LLM to check completion — yes or no?

Is there a hard maximum iteration limit set — yes or no?

Is there a tool failure exit strategy — yes or no?

Are high-stakes irreversible actions gated behind human approval — yes or no?

Has the agent been tested with a deliberately broken tool to verify graceful exit — yes or no?

If any of these is no — the agent is not production ready.

The Closing Line — Say This in the Interview

"Termination logic is where most first-time agent builders make their biggest mistake. They spend all their time making the agent work and treat stopping as an afterthought. But in production an agent that cannot stop reliably is more dangerous than an agent that never starts. The minimum viable termination design is two strategies working together

— goal achievement detection as the ideal exit, and a hard maximum iteration limit as the unconditional safety net. Every additional strategy — confidence threshold, tool error limit, human-in-the-loop — adds a layer of production resilience. The agents that survive in production long-term are not the ones that reason best. They are the ones that know when to stop."

6. What is Agentic RAG?

The One Line Opening

"Agentic RAG is what happens when you stop treating retrieval as a fixed step and start treating it as a decision. Instead of always retrieving once and answering, the agent decides when to retrieve, what to retrieve, how many times to retrieve, and from where — based on what it observes at each step."

First — Understand Plain RAG in One Paragraph

Plain RAG is a fixed two-step pipeline. User asks a question. System retrieves the most relevant documents from a vector database. LLM reads those documents and generates an answer. Done. The sequence never changes. The system never asks itself — were those documents actually relevant? Do I need more context? Should I search somewhere else? It just retrieves once and answers once regardless of quality.

What Makes RAG "Agentic"

The moment you wrap retrieval inside an agent loop — where the agent can decide to retrieve again, try a different query, search a different source, combine multiple retrievals, or stop when it has enough — you have crossed from plain RAG into Agentic RAG.

The core difference is one word — decision.

Plain RAG — retrieval is a fixed mechanical step. Agentic RAG — retrieval is a dynamic decision the agent makes based on what it observed from the last retrieval.

The 5 Things Agentic RAG Can Do That Plain RAG Cannot

1 — Multi-step Retrieval

Plain RAG retrieves once. Agentic RAG can retrieve multiple times across multiple iterations — each retrieval building on what the previous one found.

If the first retrieval returns documents that mention a concept the agent does not have enough context on, it retrieves again specifically for that concept before constructing the final answer.

2 — Query Rewriting

If the first retrieval returns poor quality or irrelevant documents, the agent observes that the results were insufficient, rewrites the query with better keywords or a different angle, and retrieves again.

Plain RAG never rewrites. Whatever the user typed is what goes into the retrieval. If the user phrased their question poorly, the retrieval is poor, and the answer is poor.

3 — Multi-Source Retrieval

Plain RAG typically searches one knowledge base. Agentic RAG can decide — based on what it observes — to search multiple sources. Internal documents first. Then a web search if internal docs are insufficient. Then a specific database if the web search raises a data point that needs verification.

4 — Retrieval Quality Evaluation

The agent reads the retrieved documents and evaluates whether they are actually relevant and sufficient to answer the goal. If they are not, it does not blindly use them. It decides to retrieve again with a better strategy.

Plain RAG has no quality gate. Whatever comes back from retrieval goes straight to the LLM regardless of relevance.

5 — Knowing When Enough is Enough

The agent decides when it has gathered sufficient context to answer confidently. It stops retrieving not because it has completed a fixed sequence but because it has evaluated the information and judged it sufficient.

Side by Side — Plain RAG vs Agentic RAG

	Plain RAG	Agentic RAG
Retrieval steps	Always exactly one	As many as needed
Query rewriting	Never	Yes — if first retrieval is poor
Multi-source search	No	Yes — agent decides which sources

	Plain RAG	Agentic RAG
Quality evaluation	No	Yes — agent evaluates each retrieval
Loop	No	Yes — full Perceive Think Act Observe
Decision making	None	Full dynamic decision at every step
Failure recovery	None	Retries with different strategy

Complete Real Example — Plain RAG vs Agentic RAG on the Same Goal

Goal — "What is Nike's current revenue growth strategy in the Asia Pacific region and how does it compare to their global strategy?"

Plain RAG handles it like this:

Step 1 — User query goes into vector database search. Step 2 — System retrieves top 3 most semantically similar documents. Returns Nike's 2022 annual report excerpt, a general article about Nike's global marketing, and a press release about a new shoe launch in China. Step 3 — LLM reads these three documents and generates an answer.

Problem — The 2022 annual report is outdated. The marketing article is about brand strategy not revenue strategy. The press release is about a product not a growth strategy. None of these directly answer the question about Asia Pacific revenue growth versus global strategy. But plain RAG does not know that. It just feeds them to the LLM anyway. The LLM produces a confident-sounding answer built on poor sources.

Agentic RAG handles it like this:

Iteration 1 Perceive — Goal is to compare Nike's Asia Pacific revenue strategy with global strategy. Think — I need recent information about Nike's Asia Pacific revenue performance. Search internal documents first. Act — Retrieves from internal knowledge base with query "Nike Asia Pacific revenue strategy." Observe — Retrieved 3 documents. Two are from 2021 — too old. One is relevant but only covers distribution channels not revenue strategy. Quality insufficient.

Iteration 2 Perceive — Previous retrieval was insufficient. Need more recent and specific data. Think — Try a more specific query focused on recent financial data. Act — Rewrites query to "Nike APLA revenue growth 2024 2025 financial results." Retrieves again. Observe — Retrieved Nike's most recent earnings report excerpt and an analyst report from last quarter. Both highly relevant. Good quality data on Asia Pacific numbers.

Iteration 3 Perceive — Have Asia Pacific data. Now need global strategy for comparison. Think — I have the regional data. Now retrieve global strategy documents separately. Act — Retrieves with query "Nike global revenue growth strategy 2024 2025." Observe — Retrieved Nike investor day presentation and global strategy memo. Sufficient for comparison.

Iteration 4 Perceive — Have both Asia Pacific data and global strategy data. Context is sufficient. Think — I have enough high-quality information from both angles to answer the goal completely. No further retrieval needed. Act — Synthesises both sets of retrieved documents into a structured comparative answer. Observe — Answer complete. Goal achieved. Loop exits.

Four iterations. Four targeted retrievals. Each one building on what the previous one found. Final answer built on high-quality relevant sources instead of whatever happened to be semantically closest to the original query.

Another Real Example — Customer Support Agentic RAG

Goal — "My order 45821 has not arrived. It has been 10 days. What happened?"

Plain RAG — Retrieves general shipping policy documents. Tells the customer "standard delivery takes 5 to 7 business days." Completely useless. It retrieved policy docs when the customer needed order-specific data.

Agentic RAG —

Iteration 1 — Retrieves order 45821 details from order management database. Observes order was dispatched 9 days ago and shows in transit.

Iteration 2 — Observes last tracking update was 6 days ago with no movement since. Decides to retrieve from logistics partner database for live tracking.

Iteration 3 — Retrieves live tracking. Observes package is stuck at a customs clearance facility.

Iteration 4 — Retrieves customs delay policy and resolution process documents. Has sufficient information now.

Final answer — "Your order 45821 is currently held at customs clearance. Based on our policy this typically resolves within 2 to 3 business days. Here is what you can do in the meantime and here is our compensation policy if it is delayed further."

Specific. Accurate. Actionable. Built on four targeted retrievals from four different sources. Plain RAG could never produce this.

Where Agentic RAG Sits on the Agentic AI Spectrum

Level 1 — Basic LLM. No retrieval. One prompt one answer. Level 2 — Plain RAG. One fixed retrieval then answer. **Level 3 — Agentic RAG. Dynamic multi-step retrieval with decision making. This is where agency begins.** Level 4 — Full AI Agent. Multiple tool types beyond just retrieval — APIs, code execution, email, database writes. Level 5 — Multi-Agent System. Multiple specialised agents collaborating.

Agentic RAG is the entry point into true agentic behaviour. It is also the most practical first step for any team that already has a RAG pipeline and wants to make it genuinely agentic without rebuilding everything from scratch.

The Closing Line — Say This in the Interview

"Plain RAG is a pipeline — retrieve once, answer once, stop. Agentic RAG is a loop — retrieve, evaluate quality, decide whether to retrieve again, rewrite the query if needed, search a different source if needed, and only answer when the agent has judged that it has sufficient high-quality context. The difference is not technical complexity — it is the presence of a decision making loop around retrieval. That loop is what makes it agentic. And that loop is what makes the answers dramatically better on any question that cannot be answered from a single perfect document."